

PROJETO LICENÇA

Hoje você deve pensar como um analista de sistema Senior que precisa pensar na solução de uma problema.

Da história:

Atualmente aqui na Aplipack, temos clientes que utilizam alguns software que fazem integração com as máquinas seja trocas de dados via TCP com integração de CPL ou não. Mas geralmente além das máquinas temos um software integrado.

Esses software de modo geral podem trabalhar de forma Offline sem a necessidade de conexão com a Internet. A instalação desses sistemas seguem um padrão de instalação isolada de forma remoto ou presencial, trata-se de uma instalação simples da aplicação em si e um banco de dados local.

Um ponto importante o que temos que e sua ativação (do Software) atualmente é realizada de forma local, existe uma variedade de software, e no geral precisamos pensar em como gerar uma ativação do software baseada nos dados de dispositivos, além de que, os software atuais possuem uma serial de ativação implícito, qual deve ser usado para compor o sistema de ativação.

Como ferramenta de controle interno para tratamentos dessas licenças de software, estamos em desenvolvimento uma ferramenta LicencaAPI, qual já está em andamento de criação.

O LicencaAPI atual foi construído nos padrões REST, possui os métodos necessários para consultas, novos ou atualizações de registros.

Com uma interface agradável para o usuário permitindo visualizar os principais dados relacionados aos equipamentos que possuem licença de software *ativos com permissão de uso, dentre os campos de dados que usuário/administrador pode ter acesso:

Tipo de Licença, * "Mac" do Dispositivo, Data, Scade e Status da Licença, Sistema Operacional, Tipo e Hostname, Descrição do Software em uso, Ip, * Processador.

O Desafio para o analista de sistema, será pensar de que forma deve-se receber as informações desses equipamentos e se comunicar com o sistema LicencaAPI (hospedado nos Domínios da aplipack.com.br).

A API deverá receber os dados enviados pelo dispositivo que possui o software da Aplipack instalado, desde que o dispositivo esteja cadastrado no sistema LicencaAPI, será possível gerar uma nova licença retornando ao dispositivos uma nova chave de licença (válida) pelo período correspondente ao tipo de Licença contratada.

Em caso de ser uma nova instalação em um dispositivo que não possui os dados cadastrados na LicencaApi, o sistema deverá tratar esse cenário cadastrando esse novo dispositivo, entanto, uma análise de controle cadastral em conjunto com o comercial deve ser realizada, afinal, em uma situação hipotética no qual o cliente possui um contrato de licenças, e essa nova instalação não condiz com seu respectivo contrato atual.

Precisa criar os passos a passos de como tratar esse recebimento dos dados, reestruturar a LicencaAPI para que possa estar pronta para receber esse dados.

Qual a melhor opção de envio do lado do cliente, afinal não ir ter acesso direto aos clientes de forma direta.

18/07/2025

Durante conversa com o Cliente, foi verificado que ela deseja algo um pouco mais simples.

No caso o LicencaAPI mantém as informações de dados de Dispositivos do cliente na Base de Dados, conforme os dados da tabela licenças, o cenário que o usuário apresentou foi, que o software do usuário estiver usando deve ter um limite de uso baseado no tempo da sua licença, para utilizar o software de forma completo o usuário deverá ter uma licença número de série ativa.

Aqui estão os possíveis cenários:

1- Cenário - Nova Instalação e adesão contratual.

- a. Supomos que esse usuário que contratou o software feche um contrato (independentemente do tipo contratual) de utilização do sistema, após a instalação do software os dados desse dispositivo são registrados na base de dados do LicencaAPI. Quando o usuário do software usar o sistema, o software deverá abrir normalmente, afinal esse dispositivo está cadastrado corretamente na LicencaAPI e a validade de sua Licença é vigente.

*contrato = conforme o contrato o cliente poderá ter um “x” número de licenças, a cada dispositivo que tenho o software instalação, decrementa-se desse número de licenças – simultânea

2- Cenário – Nova Instalação, porém dados do Dispositivo “NÃO”, Confere !

- a. Nesse cenário algum dado do Dispositivo sofreu alteração (pode ser por exemplo uma troca PlacaMae), os dados o MAC serão outro, o software não deverá abrir normalmente até sua atualização no LicencaAPI, os novos dados desse dispositivo serão recebidos na API, que deve *validar os novos dados recebido e libera o acesso ao software posteriormente.

3- Cenário – Reinstalação de Software para contratos vigentes.

- a. Após a reinstalação do Software, o sistema deverá abrir normalmente. Quando o usuário abrir o sistema, o LicencaAPI receberá as informações desse Dispositivo, e nesse caso, os dados do dispositivos são validados e a licença é vigente.

4- Cenário – Reinstalação de Software para licenças, Suspensas !.

- a. Esse cenário acontece em que o software foi suspenso conforme sua data de validade, ao usuário do software será informado sobre a suspensão e será solicitado uma nova chave de licença.
- b. O LicencaAPI, deverá validar os dados recebido desse dispositivo e deverá analisar esse dispositivo e gerar uma nova chave de licença conforme o contrato, a geração da nova chave, deverá considerar o contrato do cliente, caso ele possua ou não licenças disponíveis, acrescentar ou decrescentar na renovação do contrato.

- c. Na renovação o cliente tem a opção de atualizar o tempo de validade, bem como o numero de licenças simultâneas.

5- Cenário – Instalação de Software para dispositivos “NÃO”, Contratual !

- a. O usuário disponibilizou um novo equipamento para instalação, porem verificou-se que essa nova instalação de software dispositivo não comporta como o numero de licenças contratadas pelo Cliente.

Sobre Software:

São aplicações que em geral funcionam de forma isoladas, geralmente com um banco Local (MYSQL), essas aplicações serão modificadas em um segundo momento para que se adequem ao funcionamento do LicencaAPI.

Quanto ao tempo de validade das licenças, foi uma ideia do cliente utilizar essa data criptografa, para que seja atualizada junto com o numero de licença informado, quando a chave de licença for solicitada. Assim o software tem um controle de acesso assertivo, quanto a data de validade, em caso vencimento.

O desafio novamente fica por conta da integração entre a aplicação local, com o LicencaAPI (hospedado, no Servidor da Aplipack)

21/07/2025

Problema com a ampliação do projeto.

Preciso recriar o projeto, simplificando para um único método de consultar e retornar os dados da tabela.

23/07/2025

Entendido. A clareza do cliente sobre os cenários e a simplificação do fluxo são muito importantes.

A nova informação central é que a **LicencaAPI** agora detém a verdade sobre os dados do dispositivo e o status da licença.

A lógica de "ativação" passa a ser mais uma "validação e, se necessário, atualização" dos dados do dispositivo e status da licença, com base na comunicação do software cliente.

O ponto sobre a data de validade ser criptografada na chave de licença é crucial para a validação offline.

Vamos reestruturar a solução com base nesses novos cenários e simplificações.

- **Resumo das Novas Premissas:**

1. **LicencaAPI** é a Fonte da Verdade: Todos os dados do dispositivo e status da licença **residem na base de dados** da LicencaAPI.
2. **Chave de Licença**: O software cliente precisa de uma "**chave de licença número de série ativa**" para funcionar completamente. Esta chave conterá a **data de validade criptografada** para validação offline.
3. Contratos e Limite de Licenças: Existe um **limite 'x' de licenças por contrato**. Cada instalação do software (dispositivo) **decrementa esse número**.
4. Comunicação Cliente-API: O software cliente se comunica com a LicencaAPI para validar e, se necessário, atualizar seu estado de licença. *

Reestruturação da Solução (Foco nos Cenários e na Chave Criptografada)

A solução será novamente dividida em:

1. Melhor Opção de Envio do Lado do Cliente (Revisada).
2. Detalhes da Geração/Validação da Chave de Licença Criptografada.
3. Fluxo de Processamento de Ativação/Validação na LicencaAPI (Cenário a Cenário).
4. Reestruturação da LicencaAPI (Modificações de Código Específicas).

1. Melhor Opção de Envio do Lado do Cliente (Revisada)

A abordagem geral de comunicação via HTTP/HTTPS POST em JSON para um endpoint da LicencaAPI permanece a mais adequada.

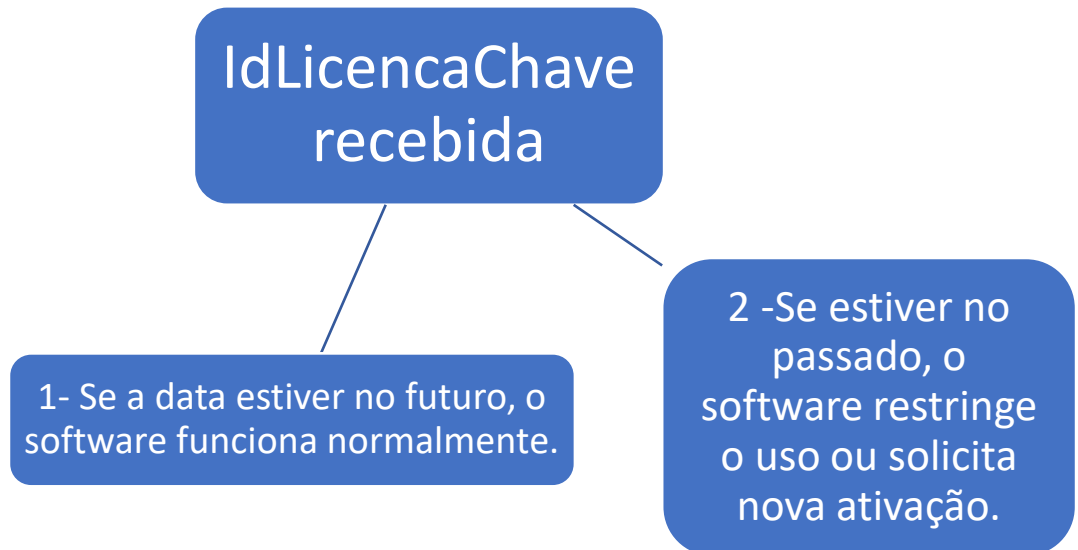
Módulo Cliente:

O software cliente terá um módulo de licenciamento responsável por:

Coletar os dados do dispositivo (**MAC Address, Processador, NomeComputador, SistemaOp, TipoPc, IP**).

1. Coletar a **IdLicencaChave** (serial implícita) que ele já possui (se houver, ou gerada na primeira instalação para "**identificação provisória**").
2. Enviar esses dados para a **LicencaAPI** em um **endpoint** de validação/ativação.

3. Receber a resposta da **LicencaAPI**, que conterá a nova **IdLicencaChave** (se a licença for válida/atualizada) e metadados (Status, Mensagem).
4. Validar a **IdLicencaChave recebida**: O software cliente deverá ter a
 - a. lógica para descriptografar/validar a data de expiração contida na **IdLicencaChave** localmente.



5. Armazenar a **IdLicencaChave** e a **DataExpiracao** (descriptografada) localmente.

- **Endpoint Único de Comunicação Cliente-API:**

POST **/api/licencas/validar-e-obter**: Este endpoint será o ponto de entrada principal para o software cliente. Ele **receberá os dados do dispositivo e retornará a IdLicencaChave** e status.

- **Frequência da Comunicação:**

1. Ao Iniciar o Software: No startup do aplicativo, ele **deve tentar se comunicar com a LicencaAPI**. Se falhar (sem internet), ele verifica **a chave de licença local**.
2. Verificação Periódica (Online): Mesmo com uma chave válida local, o software pode (e deve) **tentar se comunicar periodicamente** (ex: uma vez por semana) para **garantir que a licença não foi revogada centralmente**.
3. Validação Offline: O software sempre **valida a data de expiração criptografada na IdLicencaChave** localmente **antes de permitir o uso completo**. Se a data de expiração local for no futuro, mesmo sem internet, o software funcionará.

2. Detalhes da Geração/Validação da Chave de Licença Criptografada

Esta é a parte mais crítica para a funcionalidade offline.

- Composição da IdLicencaChave (Serial de Ativação):

A IdLicencaChave não será mais apenas uma "serial implícita", mas uma chave de licença robusta e verificável offline. Ela deve conter:

- Dados do Dispositivo Hash: Um hash (ex: SHA256) do MacAddress + Processador + NomeComputador (ou outros identificadores fortes e imutáveis). Isso vincula a chave ao hardware.
- Data de Expiração: A Scade (data de validade) da licença.
- ID da Licença no Banco: O NumLic ou outro ID único da licença na LicencaAPI.
- Hash de Validação/Assinatura: Tudo isso deve ser combinado, talvez com um "sal" (salt) secreto do servidor, e então criptografado ou assinado digitalmente (usando um par de chaves assimétricas: privada no servidor, pública no cliente) para que o cliente possa verificar sua autenticidade e integridade.

Validação (Cliente):

1. O software cliente recebe a IdLicencaChave criptografada.
2. Descriptografa a chave usando a mesma lógica (chave pública ou chave de descriptografia para AES).
3. Analisa o conteúdo: NumLic, hash dos dados do dispositivo, DataExpiração, TipoLic, Software.
4. Recalcula o hash dos próprios dados do dispositivo e compara com o hash contido na chave.
 1. Se não bater, a chave é inválida para este hardware.
5. Compara a DataExpiração contida na chave com a data/hora atual.
 1. Se DataExpiração for no passado, a licença expirou.

3. Fluxo de Processamento de Ativação/Validação na `LicencaAPI` (Cenário a Cenário)

6. O método `ProcessarAtivacaoDispositivoAsync` no `LicencaService` será o  **da lógica**, manipulando todos os cenários.
7. **`AtivacaoDispositivoRequestDTO`** (já definido na resposta anterior):
`MacAddress`, `Software`, `IdLicencaChave` (chave atual do cliente, se houver), `Processador`, `SistemaOp`, `TipoPC`, `NomeComputador`, `Ip`, `IdCliente` (opcional).
8. **`AtivacaoDispositivoResponseDTO`** (já definido na resposta anterior):
`ChaveLicenca` (a nova ou validada), `DataExpiracao`, `StatusLicenca` ("Ativa", "Pendente Analise", "Expirada", "Invalida", "Sem Licencas Disponiveis"), `Mensagem`.

4. Reestruturação da `LicencaAPI` (Modificações de Código Específicas)

As modificações de código seguirão o plano anterior, com os seguintes aprimoramentos e adições:

1. **Novos DTOs:** Manter `AtivacaoDispositivoRequestDTO` e `AtivacaoDispositivoResponseDTO` conforme definido.
2. **`LicencaModel.cs`:**
 - Manter todos os campos existentes.
 - **`IdLicencaChave`:** Este campo no `LicencaModel` será agora o local para **armazenar a chave de licença criptografada** gerada pelo servidor.
 - **Adicionar `Status` (string):** Essencial para controlar o fluxo dos cenários (Ativa, Inativa, Expirada, Pendente Analise, Rejeitada, Suspensa). Isso dá mais flexibilidade que apenas `Attivo`.

```
// LicencaApi/Models/LicencaModel.cs
```

```
public string Status { get; set; } = "Pendente Analise"; // Valor padrão
```

- **Adicionar `IdContrato` (int?):** Crucial para o Cenário 1 e 5. Precisamos saber a
 1. **qual contrato a licença está vinculada para gerenciar o número de licenças simultâneas.**

```
// LicencaApi/Models/LicencaModel.cs
```

```
public int? IdContrato { get; set; }
```

- **Adicionar `NomeComputador` e `TipoPc`:** Já estão nos DTOs, mas verificar se estão no Model e mapeados corretamente. New Project 20250717 1453.sql mostra `Nome_Computador` e `Tipo_Pc`. Use a nomenclatura consistente (`NomeComputador`, `TipoPc`) e mapeie para as colunas do DB.

Nova Classe/Serviço para Geração de Chave:

- `LicencaKeyGenerator.cs` (ou `LicencaCriptoService.cs`) contendo a lógica de `GenerateSha256Hash`, `EncryptWithAES` e `DecryptWithAES`. Este serviço será injetado ou instanciado no `LicencaService`.

Atualização da Camada de Repositório (Repositories):

- `ILicencaRepository.cs` / `LicencaRepository.cs`:
 - Adicionar método para buscar licença por MAC e Software:

```
// ILicencaRepository.cs
```

```
Task<LicencaModel?> BuscarPorMacAndSoftwareAsync(string macAddress, string software);
```

```
// LicencaRepository.cs
```

```
public async Task<LicencaModel?> BuscarPorMacAndSoftwareAsync(string macAddress, string software)
{
    return await _context.Licencas
        .FirstOrDefaultAsync(l => l.MacAddress == macAddress && l.Software == software);
}
```

- Adicionar método para buscar licença por `IdLicencaChave` (se a chave implícita antiga for um identificador para migração):

```
// ILicencaRepository.cs
```

```
Task<LicencaModel?> BuscarPorIdLicencaChaveAsync(string idLicencaChave);
```

```
// LicencaRepository.cs
```

```
public async Task<LicencaModel?> BuscarPorIdLicencaChaveAsync(string idLicencaChave)
{
    return await _context.Licencas
        .FirstOrDefaultAsync(l => l.IdLicencaChave == idLicencaChave);
}
```

Considerar a criação de um repositório para `AcessosNew` se quiser gerenciar via EF Core:

- `IAcessoNewRepository.cs` e `AcessoNewRepository.cs`.

Atualização da Camada de Serviço (Services):

- **ILicencaService.cs:** Adicionar `Task<AtivacaoDispositivoResponseDTO>`
`ProcessarAtivacaoDispositivoAsync(AtivacaoDispositivoRequestDTO request);`
- **LicencaService.cs:** Implementar `ProcessarAtivacaoDispositivoAsync` com a lógica detalhada para os cenários (conforme explicado no item 3). Injetar `LicencaKeyGenerator` ou instanciá-lo.
 - Incluir lógica para interagir com o controle de Contratos e licenças simultâneas (idealmente um novo serviço `IContratoService` ou diretamente no `LicencaService` se for simples).

Atualização da Camada de Controlador (Controllers):

- **LicencaController.cs:** Adicionar o endpoint [`HttpPost("ativar")`] que delega a lógica para `_service.ProcessarAtivacaoDispositivoAsync`.

LicencaDbContext.cs:

- Mapear o novo campo `Status` em `LicencaModel`.
- Mapear `AcessoNewModel` se for gerenciar via EF Core.

Program.cs:

- Registrar o novo serviço `LicencaKeyGenerator` (ou `LicencaCriptoService`).
- Garantir que os novos repositórios (ex: `AcessoNewRepository`) sejam registrados no IoC Container.
- Garantir que a string de conexão está correta para o banco licencas.